cpw2000 User Guide

Version 5.13, Draft October 20, 2026

José Luís Martins
Departamento de Física, Instituto Superior Técnico, Lisboa, Portugal
(Retired)

INESC MN, Lisboa, Portugal

jlmartins@inesc-mn.pt
jose.l.martins@tecnico.ulisboa.pt

Contents

1 Introduction **3**

2 Installation **3**

2.1 Step 1: Downloading and extracting the archive . . . 3

2.2 Step 2: Generating the documentation . . . 3

2.3 Step 3: Compiling the code . . . 4

3 First time run of the code **4**

3.0.1 Files required to run the code . . . 5

3.1 Running the code . . . 5

4 The `cpw.in` file **6**

4.1 General stuff . . . 7

4.2 Crystal descriptors . . . 8

4.3 Major Self Consistent Field parameters . . . 9

4.4 Minor Self Consistent Field parameters . . . 12

4.5 Unused Self Consistent Field parameters . . . 12

4.6 Molecular dynamics parameters . . . 13

4.7 Symmetry . . . 16

4.8 Pseudopotential files . . . 16

4.9 Saving wave-functions to disk . . . 17

4.10 Unfolding . . . 17

5 Default output **18**

5.1 Initial information . . . 18

5.2 Crystal data . . . 19

5.3 Crystal symmetry and reciprocal space info . . . 21

5.4 The SCF cycle . . . 21

5.5 Potential, energies and forces . . . 22

6 Other files from cpw.exe **24**

6.1 The `PW_RHO_V.DAT` file . . . 24

7 Post processing **26**

7.1 Band structures, density of states, optical response, quantum geometric,... . . . 27

7.1.1 Band structure . . . 28

7.1.2 Prepare file for density of states . . . 31

7.1.3 Properties for single k-vector . . . 31

7.1.4 Prepare file for dielectric function calculation . . . 31
7.1.5 Calculate effective band masses . . . 31
7.1.6 Calculate quantum geometric quantities . . . 32
7.1.7 Reset dual approximation flag . . . 36
7.2 Modified tight binding . . . 36
7.3 Plot of charge densities and potentials . . . 37
7.4 Plot densities of states . . . 37
7.5 Exploration of a k.p model . . . 37
7.6 Plot dielectric function . . . 37
7.7 Plot of the wave-functions . . . 37
7.8 Crystalography . . . 38
7.9 Interface to Quantum Espresso . . . 38

8 Where to find crystal structures **38**

9 Toolbox **38**
9.1 Equation of state . . . 38
9.1.1 Compiling and running . . . 39
9.1.2 The `eqst.dat` file . . . 39
9.1.3 Output . . . 41
9.2 Remove gnuplot commands from 2D plot files . . . 42
9.3 Crystalography from cpw.in . . . 42
9.4 Coulomb potential . . . 42
9.5 Graphical interface for analysis of band structure . . . 43

1 Introduction

This is the current version of a old code that calcultes the electronic structure of crystals within the pseudopotential approximation with a plane-wave basis.

The very original code was written by Sverre Froyen at the University of California Berkeley. I do not know what was the first time his version of the code, as there was already an earlier pseudopotential code at Berkeley, I would guess that Physical Review B, 26, 3258 (1983) https://journals.aps.org/prb/abstract/10.1103/PhysRevB.28.3258 already used that version.

After a long conversation with Roberto Car about the seminal Car-Parrinello work, I wrote at Berkeley in Marvin Cohen's group the code for the iterative diagonalization of the plane-wave matrices Physical Review B 37, 6134 (1988).

Since then many people contributed to the code. At Minnesota Norm Troullier and the group of Jim Chelikowsky contributed to the development. Renata Wentzcovitch implemented the original molecular dynamics and the variational cell shape molecular dynamis. Nadia Binggeli implemented the Langevin molecular dynamics. At Lisbon the largest contribution was from Carlos Loia Reis.

2 Installation

2.1 Step 1: Downloading and extracting the archive

The code is available from GitHub

`https://github.com/jlm785/cpw2000`

If you downloaded the `cpw2000-5.x.y.tar.gz` file (where `x` and `y` are the minor version numbers) just extract it,

```
$ tar xzf cpw2000-5.x.y.tar.gz
```

and you will have a `cpw2000-5.x.y` directory

If you cloned the git you already have the relevant code in the main directory.

2.2 Step 2: Generating the documentation

The documentation is in the `cpw2000-5.x.y/Doc` directory. It include this file, and the means to generate a detailed description of the code for developers.

For that detailed description you need doxygen (`https://www.doxygen.nl`) and graphviz (`https://graphviz.org/`), both are available on most distributions. To check that your computer has doxygen and graphviz installed run the commands

```
cpw2000-5.x.y/Doc$ dpkg -s doxygen
```

```
cpw2000-5.x.y/Doc$ dpkg -s graphviz
```

on a Debian based distribution (Ubuntu et al), or use equivalent tools (yum, rpm, dnf zypper,...) or the relevant GUI.

If they are not available just install them

```
cpw2000-5.x.y/Doc$ sudo apt install doxygen
```

```
cpw2000-5.x.y/Doc$ sudo apt install graphviz
```

again for Debian based distros, or use equivalent tools (yum, rpm, dnf zypper,...) or the relevant GUI.

Finally run doxyfile in the doxy directory

```
cpw2000-5.x.y/Doc/doxy$ doxygen Doxyfile
```

If you know what you are doing you can edit `cpw2000-5.x.y/Doc/doxy/Doxyfile`.

Finally it is useful to create a link to the file `cpw2000-5.x.y/Doc/html/index.html`

```
cpw2000-5.x.y/Doc$ ln -s html/index.html documentation.html
```

Opening that link in your browser will allow you to see the documentation for (almost) every file in the code.

2.3 Step 3: Compiling the code

The code has been tested with several Fortran compilers, `ifort` (from Intel oneAPI), `gfortran` (from gnu), `pgfortran` (from Portland), and even LLVM (experimental Intel compiler).

The choice of compiler is in the `cpw2000-5.x.y/Src/make.inc` file. You may edit it for your convenience, but in principle you have to make just two choices. The first is to identify which CPU you are using, as it narrows the choice of possible compilers. This is done by commenting/uncommenting the lines below `# compiler for the job`.

The second is the compiler you want, and is the key decision. Again comment/uncomment the lines below `# Suggestions for compilers`.

For any other modification of the `make.inc` file it is assumed you know what you are doing... The `make.inc` file was adapted from the WANNIER90 code `https://wannier.org/`. As compilers may evolve with time, if you run into problems check the documentation and corresponding textttmake.inc file from that distribution. Another code that uses a `make.inc` file is ELK `https://elk.sourceforge.io/`. There you can find suggestions for compiler options.

To compile just go to `cpw2000-5.x.y/Src` and type make

```
cpw2000-5.x.y$ cd Src
```

```
cpw2000-5.x.y/Src$ make
```

You should get the main library `libcpw_`*compiler*`.a` and a few executables `cpw_`*compiler*`.exe`, etc... where *compiler* is the name of the chosen compiler. As you probably are not interested in comparing compilers, it is advisable to create links without the compiler name. For example, after compiling with `ifort`, I usually create the links

```
cpw2000-5.x.y$ ln -s Src/cpw_ifort.exe cpw.exe
```

```
cpw2000-5.x.y$ ln -s Src/cpw_post_process_ifort.exe cpw_post_process.exe
```

```
cpw2000-5.x.y$ ln -s Src/libpw_ifort.a libpw.a
```

In the examples it will be assumed that those links are created, but obviously the links could be anywhere (in a `bin` folder for example).

3 First time run of the code

I assume you have a working directory separate from the source. In the following I will call it `cpw2000-5.x.y/WORK`

3.0.1 Files required to run the code

You will need a file with the crystal description and a pseudopotential file for each type of atom in the crystal.

The file with the crystal description is called `cpw.in`. You must have such a file in your working directory. You can find such a file for almost all elements in `cpw2000-5.x.y/Structures/Elements` and files for other structures in the other subdirectories of textttcpw2000-5.x.y/Structures. The format of the crystal description is the same as in the SIESTA code, textttthttps://siesta-project.org/siesta/index.html so you may find descriptions for other crystals in the "net". **Beware** that presently the code only accepts **fractional** lattice coordinates.

The old fixed format `PW.DAT` file is no longer accepted. It can be converted with the tool `Tools/pwdat2cpwin.f90`, which writes the file `cpw_from_pwdat.in`. If the old `cpw.in` file used with `PW.DAT` is present, the type of run, the Keating correction and the TBL choice of exchange and correlation are taken from it, the other options have to be copied by hand.

There is also a tool, `Tools/gen_PW.f90`, to obtain a `cpw.in` file by answering a few questions.

You will have to generate the pseudopotentials for each atom, or find someone who has done that for you and have a file in the relevant format. The name of the file is `Xy_POTKB_F.DAT` where `Xy` is the one or two character chemical symbol. This file should be (or at least a link) on your working directory, or an explicit path should be specified in `cpw.in`.

The good news is that the code to generate pseudopotentials is available from GitHub

`https://github.com/jlm785/pseudopotential`

and if you run the test on `Validation` after you followed the relevant instructions to install the code you will get pseudopotential files for all elements.

The bad news is that some of those pseudopotentials where not extensively tested, and even those that are reported as tested may not be what you want. Just test them in a simple case before you proceed. As pseudopotentials calculations should reproduce first principles calculations in `cpw2000-5.x.y/Structures/Elements` you can find `elk.in` files that you can use to run LAPW all-electron calculations with the ELK code, `https://elk.sourceforge.io/`. Testing the pseudopotential will spare you a lot of grief later.

If you want just a "quick and dirty" calculation than you can use the pseudopotentials from `Validation` of the pseudopotential code.

3.1 Running the code

Once you have the required files you run the code with

`cpw2000-5.x.y/WORK$ ../cpw.exe`

if you created the appropriate links, or

`cpw2000-5.x.y/WORK$ ../Src/cpw_`*compiler*`.exe`

otherwise. The code will write to default output, so redirect it with

`cpw2000-5.x.y/WORK$ ../cpw.exe > output.dat`

or

`cpw2000-5.x.y/WORK$ ../cpw.exe | tee output.dat`.

You will get a `PW_RHO_V.DAT` file with the self-consistent potential. By running

```
cpw2000-5.x.y/WORK$ ../cpw_post_process.exe
```

you will start an interactive analysis of the results.

If both executables ran without errors you are set.

4 The `cpw.in` file

The `cpw.in` syntax is similar to the input file from the SIESTA code. The parser is different, `esdf` instead of `fdf`. The keywords are even the same when possible. Therefore part of the text of this section was pilfered from the SIESTA documentation.

The `cpw.in` contains all the physical data of the system and the parameters of the simulation to be performed.

This file is written in a special format called ESDF, developed by Chris J. Pickard. This format allows data to be given in any order, or to be omitted in favor of default values. It is used in other electronic structure codes such as CASINO and PARSEC. However to be consistent with SIESTA (which uses the fdf format) the special character indicating a block structure has been changed.

Here we offer a glimpse of the rules:

- The syntax is a 'data label' followed by its value. Values that are not specified in the datafile are assigned a default value.
- The labels are case insensitive, and characters - _ . in a data label are ignored. Thus, LatticeConstant and lattice_constant represent the same label.
- All text following the # character is taken as comment.
- Logical values can be specified as T, true, .true., yes, F, false, .false., no.
- Character strings should **not** be in apostrophes.
- Real values which represent a physical magnitude must be followed by its units. It is important to include a decimal point in a real number to distinguish it from an integer, in order to prevent ambiguities when mixing the types on the same input line.
- Complex data structures are called blocks and are placed between '%block label' and a '%endblock label' (without the quotes).
- If the same label is specified twice, the first one takes precedence.
- If a label is misspelled it will not be recognized (there is no internal list of "accepted" tags in the program). You can check the actual value used by CPW2000 by looking for the label in the output (by default standard output).

This is an example for silicon:

```
LatticeConstant             10.2629     bohr

%block LatticeVectors
```

```
         0.00000000         0.50000000         0.50000000
         0.50000000         0.00000000         0.50000000
         0.50000000         0.50000000         0.00000000
%endblock LatticeVectors

NumberOfSpecies               1

NumberOfAtoms                2

%block Chemical_Species_Label
               1       14   Si
%endblock Chemical_Species_Label

AtomicCoordinatesFormat     Fractional

%block AtomicCoordinatesAndAtomicSpecies
         0.12500000      0.12500000      0.12500000      1     #  Si       1
        -0.12500000     -0.12500000     -0.12500000      1     #  Si       1
%endblock AtomicCoordinatesAndAtomicSpecies

StructureSource               Experiment

#-----------------------------------------------
# Energy cutoff, bands,  and Brillouin mesh
#-----------------------------------------------

PWEnergyCutoff                  12.0000      hartree

NumberOfEigenStates                 10

%block kgrid_Monkhorst_Pack
               4       0       0       0.500000
               0       4       0       0.500000
               0       0       4       0.500000
%endblock kgrid_Monkhorst_Pack
```

4.1 General stuff

- **SystemLabel**

  A *single* word (max. 20 characters *without blanks*) containing a nickname of the system. Reserved to be used to name output files in the future.

- **PrintingLevel** (integer)

  Defines the detail of the printout. May take the values 1,2, or 3.

  Default value: 1 (for molecular dynamics) or 3 (single geometry).

  The higher the value the more details will be printed. Remember that too much information is noise. 1 is recommended for molecular dynamics. 3 for single calculation, or when things seem to go wrong.

- **MD.TypeOfRun** (string)

  Choice on how the atoms move. Molecular dynamics or geometry optimization.

  Default value: ONE

  With the default it will run a single geometry. See below for the other options to run molecular dynamics calculations.

  The value in `cpw.in` may be overridden by the first argument of the executable.

  ```
  cpw2000-5.x.y/WORK$ ../cpw.exe MICRO > output.dat
  ```

  will perform a microcanonic molecular dynamics calculation irrespective of the value in `cpw.in`.

4.2 Crystal descriptors

These are the lines that describe the crytal structure.

- **LatticeConstant** (length)

  A physical value, requiring a real number followed by the units. Accepted units are: `bohr, ang, nm, m`. This is just to define the scale of the lattice vectors.

  Default value: 1.0 bohr.

  The code stops if the units are not present or not recognized. Internally the code uses atomic units (bohr).

- **LatticeVectors** (block)

  The cell vectors are read in units of the lattice constant defined above. They are read as a matrix, each vector being one line.

  Default value: Unit matrix.

  The internal representation of the lattice vectors in the code is by their metric tensor. So the original orientation is space is lost! When the code needs to read or print information with orientation content it will use some *canonic* vectors based on the lattice symmetry. Those are printed at the start of the SCF code, and before questions to the user in the post-processing code. Just pay attention, they may be **different** from what is written in `cpw.in`! See the section on output for more details.

  The code checks if the entered value is near a rational number or the square root of a rational number (with low denominators). If it is the case it will use that modified value and print a warning. This allows to have an exact symmetry, and the modifications are always small.

- **NumberOfSpecies** (integer)

  Number of different atomic species in the simulation. It must be the number of lines in the **ChemicalSpeciesLabel** block.

  Default value: 1

  If not present or nin positive it will use the number of lines in **ChemicalSpeciesLabel**.

  In case of inconsistency the execution terminates.

- **NumberOfAtoms** (integer)

  Number of atoms in the simulation. It is the number of lines in the block **AtomicCoordinatesAndAtomicSpecies**.

  Default value: 1

  In case of inconsistency the execution terminates.

- **ChemicalSpeciesLabel** (block)

  It specifies the different chemical species that are present, assigning them a number for further identification. cpw2000 recognizes the different atoms by the given atomic number.

  One line for each species. The first number in a line is the species number, it is followed by the atomic number, and then by the chemical symbol. From H to Og all chemical symbols are recognized. There is an extra chemical symbol, ZZ with a number of protons (atomic number) that can be zero to allow adding special pseudopotentials to the crystal.

  In case of inconsistency the execution terminates.

- **AtomicCoordinatesFormat** (string)

  Character string to specify the format of the atomic positions in input. **It is not used!!!** It is here for future use and compatibility with SIESTA.

- **AtomicCoordinatesAndAtomicSpecies** (block)

  Block specifying the position and species of each atom. One line per atom, with three reals followed by one integer. In total the number of lines indicated by **NumberOfAtoms** must be present.

  The three reals indicate the atomic positions in **FRACTIONAL lattice coordinates** followed by the species of atom on that position, as identified in the **Chemical_Species_Label** block.

  In case of inconsistency the execution terminates.

- **StructureSource** (string)

  Information on the source of the crytal structure. Not used by the code,

4.3 Major Self Consistent Field parameters

This are the parameters whose values you would mention in a paper to allow reproducibility.

- **PWEnergyCutoff** (energy)

  Energy cutoff of the plane wave basis set expansion. Real value followed by the energy unit. Accepted units are: `eV, Ry, Hartree`, and a few other.

  Default value: 5 Hartree

  Internally the code uses Hartrees.

- **NumberOfEigenStates** (integer)

  This parameter indicates the number of eigenstates to be calculated.

  Default value: 10

  In some future the default should be slightly larger than half the number of electrons.

By choosing a low value the cost of the diagonalization may be reduced by finding fewer eigenstates. However choosing a slightly larger number of active eigenstates than the bare minimum may help converge faster the occupied eigenstates and therefore the overall calculation. Note, that if the electronic temperature is greater than zero then the number of partially occupied states increases, depending on the band gap. The value specified must be greater than the number of occupied states (at least the number of electrons divided by two for a non-spin-polarized calculation) and less than the number of basis functions (which is extremly large for plane waves).

- **kgridMonkhorstPack** (block)

  Specifies the Fourier integration grid, known in the literature as the Monkhorst-Pack grid, for the Brillouin zone integration. It is just the good old Gauss quadrature method with sine/cosine functions chosen for periodic functions.

  Default value: The $4 \times 4 \times 4$ sampling with 1/2 shift.

$$\begin{array}{cccc} 4 & 0 & 0 & 0.5 \\ 0 & 4 & 0 & 0.5 \\ 0 & 0 & 4 & 0.5 \end{array}$$

  Specified as an integer 3×3 matrix and a real vector. At present only the diagonal elements of the matrix are relevant, but this format will alow an extension of the method in the future and better compatibility with SIESTA.

  It has three lines, each with three integers and a real.

$$\begin{array}{cccc} m_{11} & m_{12} & m_{13} & d_1 \\ m_{21} & m_{22} & m_{23} & d_2 \\ m_{31} & m_{32} & m_{33} & d_3 \end{array}$$

  In the direction j the Brillouin zone will be divided in m_{jj} sections and a point with a shift of d_j will be selected.

  To use only the Γ point (molecule in a supercell) use

$$\begin{array}{cccc} 1 & 0 & 0 & 0.0 \\ 0 & 1 & 0 & 0.0 \\ 0 & 0 & 1 & 0.0 \end{array}$$

  It is usual to have m_{jj} an even number and $d_j = 0.5$. For hexagonal crystals and the conventional axis, it is also usual/convenient to have $m_{11} = m_{22}$ a multiple of three. These are recipes that minimize the number of irreducible points and avoid breaking symmetry.

  If the diagonal elements are non-positive it will default to $m_{ii} = 1$, printing a warning. It also warns the presence of unused non-zero off-diagonal values.

- **XC.Authors** (string)

  Particular parametrization of the exchange-correlation functional.

  Default value: CA

  - CA

    Local density approximation (LDA). Quantum Monte Carlo calculation of the homogeneous electron gas by D. M. Ceperley and B. J. Alder, Phys. Rev. Lett. **45**,566 (1980), as parametrized by J. P. Perdew and A. Zunger, Phys. Rev B **23**, 5075 (1981)

- PBE

  GGA of J. P. Perdew, K. Burke and M. Ernzerhof, Phys. Rev. Lett. **77**, 3865 (1996)

- TBL

  Meta-GGA of Tran and Blaha. F. Tran and P. Blaha, Phys. Rev. Lett. 102, 226401 (2009)

- **Xc.TBL.C** (real)

  Sets Tran-Blaha constant if a positive value. If it is negative, the original constant is used. A value around 1.09 is usualy a good choice. It can be used to "fine-tune" the band gap in simulations (second-priciples calculations).

  Default value: 1.0

- **TypeOfScfDiag** (string)

  Indicates how the SCF was performed.

  - PW

    The full plane-wave basis set. It is the "usual" or traditional choice.

  - AO

    Uses the atomic orbitals included in the pseudopotential file. Not available in very old files. If the file has several atomic basis sets the choice is made with `ChoiceOfDefaultAtomicBasisSet`. It corresponds to a Linear Combination of Atomic Orbitals (LCAO) calculation. It is very fast, but with limited accuracy. Use for *exploratory* runs on complex structures.

  - AOJC

    Uses the atomic orbitals included in the pseudopotential file, but improves the LCAO wave-functions with a single iteration Jacobi correction. Slower than the AO option but still quite fast. It is very convenient to explore the Born-Oppenheimer energy surface in optimization and molecular dynamics. However one may want to check the *final* results with a full PW calculation.

  - AOJCPW

    Follows an AOJC calculation with a full PW calculation. Final results are the practically the same as the normal PW calculation, but may be faster. Check if that is the case for your crystals before using..

- **DualApproximation** (boolean)

  Use the dual approximation.

  Default value: `.TRUE.`

  It uses a smaller grid for the calculation of the effective potential. Speeds up the calculations with a compromise on precision. For molecular dynamics it is very safe. For geometry optimization with small energy cutoffs may not be accurate enough.

- **ElectronicTemperature** (temperature)

  A real value followed by unit. The normal unit is the Kelvin, `k`, but will accept other energy units, for example `meV`

  Default value: 0 K

  Occupy orbitals with a Fermi-Dirac distribution with that temperature.

- **TypeOfPotentialMixing** (string)

  Default value: `BFGS`

  Type of effective potential mixing used to accelerate convergence. BFGS is Broyden-Fletcher-Goldfarb-Shanno method. It has some modificatitions for the maximum and minimum values of mixing. BROYD1 is Broyden 1st method. In a few cases it may be faster, but in most cases is slower, and for difficult cases (supercells) it may not converge. It has not been tuned in a long time.

4.4 Minor Self Consistent Field parameters

Other parameters of self-consistency that have less impact, unless "very wrong" choices are made. Probable safe to leave at default values.

- **MaxSCFIterations** (integer)

  Maximum number of self-consistent iterations.

  Default value: 30

  If very large, computing time can be wasted in the cases convergence is not achieved. If very small the code can terminate without a result, also wasting computer time.

- **ScfTolerance** (real)

  Convergence parameter (atomic units/Hartree) for self-consistency

  Default value: 0.00001

  The SCF iterations are deemed converged if the difference between the input and output values of all the components of the Fourier transform of the effective potential are smaller than this treshold.

  For exploratory runs decrease this parameter.

- **MaxDiagIterations** (integer)

  Maximum number of Ritz steps in the iterative diagonalization subroutine.

  Default value: 40

  Increase if in the last few SCF iterations you still see warnings about noisy diagonalization. In early SCF iterations, those warnings disappear if you increase this parameter, but computing time will also increase without changing final results.

- **DiagTolerance** (real)

  Criteria for iterative diagonalization convergence.

  Default value: 0.00001

  The iterative diagonalization is deemed converged if the module of the error vector is smaller than that value. For exploratory runs you can use the value of 0.001.

4.5 Unused Self Consistent Field parameters

Over the years only one option was left in the code.

- **TypeOfPseudopotential**

  Type of pseudopotential that is used.

  Default value: `PSEUKB`

  Kleinman-Bylander separable pseudos are the only available option. In older versions, the non-separable pseudo was available (PSEUDO), as well as a gaussian integration separable pseudopotential (PSEUGA). Some subroutines and variables still keep the `ga` characters from those days.

4.6 Molecular dynamics parameters

- **MD.TypeOfRun** (string)

  Choice on how the atoms move. Molecular dynamics or geometry optimization.

  Default value: ONE

  - ONE

    Just one SCF calculation. Atoms do not move.

  - RSTRT

    Restart the molecular dynamics from the last atomic configuration. Use in case it was interrupted by some external reason (power failure). Just keep all the other parameters the same. It will reproduce what a non-interrupted calculation would find in the case of molecular dynamics. For optimization the minimization restarts, so it will not reproduce a non-interrupted calculation, but probably will converge to the same minimum.

  - MICRO

    Microcanonic molecular dynamics. The total/free energy is conserved, depending whether the temperature is zero and the system is an insulator or temperature is finite and not too low in the case of a metal. How low the temperature can go depends on the density of integration k-points.

  - LANG

    Langevin molecular dynamics, in contact with a thermostat with temperature indicated by **MD.TargetTemperature**.

  - VCSLNG

    Langevin molecular dynamics but with a variational cell shape, that is the lattice vectors change with simulation time.

  - LBFSYM

    Minimization of the energy with respect to atomic positions with the LBF algorithm.

  - VCSLBF

    Minimization of the energy with respect to atomic positions and cell shape with the LBF algorithm.

  - EPILBF

    Minimization of the energy with respect to atomic positions and "vertical" dimension of the cell with the LBF algorithm. It models an epitaxial situation. The initial $\vec{a}_1$ and $\vec{a}_2$ of **LatticeVectors** define the epitaxial surface. The cell only contracts and expands in the direction perpendicular to those vectors ($\vec{b}_3$).

- **MD.InitialTemperature** (temperature)

  A real value followed by unit. The normal unit is the Kelvin, `k`, but will accept other energy units, for example `meV`

  Default value: 1000 K

  The initial kinetic energy is set according to the temperature . If the initial potential energy is high, the temperature of the system may be quite higher. If unsure, first do a thermalization with Langevin, otherwise the system may break apart. Simulations with **TypeOfScfDiag** as AO or AOJC are great for fast thermalization.

- **MD.NumberOfSteps** (integer)

  Number of steps of the molecular dynamics run or maximum number of optimization steps.

  Default value: 100

- **MD.LengthTimeStep** (time)

  A real value followed by unit. The usual unit are femtoseconds, `fs`, but will accept other time units `s, ns, ps, autime`.

  Default value: 2.418884 `fs` = 100.0 `autime`

  Time step for molecular dynamics. If ut is too small atoms barely move, if too large the integrator of the molecular dynamics (Verlet) becomes unstable. Instability appears in the non-conservation of energy. Few femtoseconds should be OK, but remember that light atoms move faster (square root of mass scaling). Simulations with **TypeOfScfDiag** as AO or AOJC are great for initial checks.

- **MD.TargetTemperature** (temperature)

  A real value followed by unit. The normal unit is the Kelvin, `k`, but will accept other energy units, for example `meV`

  Default value: 300 K

  For molecular dynamics with a Langevin thermostat, it is the temperature of the thermal bath.

- **MD.FrictionFracInvTimeStep** (real)

  The friction coefficient for simulations with a Langevin thermostat.

  Default value: 20.0

  It is set with respect to the time step. It indicates how many time steps occur until some thermalization is achieved. If small the thermalization is of bad quality, if large it will take a long time to thermalize.

- **MD.TargetPressure** (pressure)

  A physical value, requiring a real value followed by unit. Accepted units are: `GPa, Pa, atm, Mbar, bar, MPa`.

  Default value: 0 `GPa`

  In a molecular dynamics with variational cell shape, or cell optimiztion, it is the applied pressure.

- **MD.TargetStress** (block)

  Real 3×3 matrix of an applied stress in addition to the applied pressure. The units are in GPa only.

Default value: 0

The matrix should be symmetric, and is in lattice coordinates, so some effort is needed to understand the orientation of the crystal.

- **MD.CellMass** (real)

  Fictitious cell mass for variational cell shape molecular dynamics. It is in units of electron mass.

  Default value: 10.0

  Should be fairly larger than one for the cell dynamics to be slow, but not so large that the cell dynamics is too slow. Simulations with **TypeOfScfDiag** as AO or AOJC are great for initial checks.

- **MD.Seed** (integer)

  Seed for the pseudo-random generator for the thermostat and iniial velocity.

  Default value: 87697

  Same seed will give the same trajectory. Different seeds will give different trajectories allowing trivial parallelization of simulations.

- **MD.UseKeatingCorrections** (boolean)

  Default value: `.false.`

  For some tetrahedral semiconductors, it provides a correction to LDA that reproduces experimental bond lengths. Do not use with heavy elements (Sn). If the structure is not tetrahedral the code stops.

- **MD.UseFixedkplusG** (boolean)

  Default value: `.false.`

  Keep a fixed $\vec{k}+\vec{G}$ expansion in a variational cell shape simulation to avoid the noise of the basis set changing during the simulation. Should only be used near equilibrium. abinit has a neater way of dealing with this noise, the code is prepared to use that trick, but it has not been implemented. Use **MD.CG.FixedkplusGTol** to define when it kicks in.

- **MD.CG.Tolerance** (force)

  A physical value, requiring a real value followed by unit. The normal and internal unit is `har/bohr`, but will accept `N, eV/ang, Ry/bohr`.

  Default value: 0.0001 `har/bohr`

  A geometry optimization run stops when the components of the forces on all atoms are smaller than this value.

- **MD.CG.StepMax** (length)

  A physical value, requiring a real value followed by unit. Accepted units are: `bohr, ang, nm, m`.

  Default value: 0.01 `bohr`

  Maximum displacement of atoms to avoid instabilities in the optimization.

- **MD.CG.FixedkplusGTol** (force)

  A physical value, requiring a real value followed by unit. The normal and internal unit is `har/bohr`, but will accept `N, eV/ang, Ry/bohr`.

Default value: 0.01 `har/bohr`

The fixed $\vec{k} + \vec{G}$ is used *after* the components of the forces (har/bohr) on all atoms are smaller than this value.

4.7 Symmetry

Treatment of symmetry during a molecular dynamics or optimization run.

- **UseSymmetry** (boolean)

  Default value: `.TRUE.` for optimization and single runs, `.FALSE.` for molecular dynamics runs.

  Try to maintain the initial symmetry of the system. It is close to 100% fiable (as close as real number logic allows).

- **SymmTolerance** (real)

  Tolerance for symmetry identification.

  Default value: 0.00001

  If after a symmetry operation the difference between the positions (in lattice coordinates) is smaller than this value, the atoms are considered superposed.

4.8 Pseudopotential files

Where to find the pseudopotential files. By default they should be on the directory where the code is run. The default name of the file is `Xy_POTKB_F.DAT` where `Xy` is the one or two character chemical symbol.

- **PathToPseudos** (string)

  Directory where the pseudopotentials are found, either full specification, or starting from the running directory

  Default value: null string, indicating running directory.

- **PseudoSuffix** (string)

  Suffix to add to to the chemical symbol to obtain the file name.

  Default value: `_POTKB_F.DAT`

  It the PathToPseudos is `tmp` and the PseudoSuffix is `_POT.DAT` the file used for Si would be `tmp/Si_POT.DAT`.

- **ChoiceOfDefaultAtomicBasisSet** (block)

  Recent pseudopotential files may contain more than one set of atomic orbitals (atomic basis sets). This block chooses which one is used for each species.

  Default value: the first atomic basis set for all species.

  One line for each species whose choice differs from the default. The first number in a line is the species number, as in `ChemicalSpeciesLabel`, it is followed by the index of the atomic basis set. Species that are not listed use the first basis set.

```
%block ChoiceOfDefaultAtomicBasisSet
     2     2
%endblock ChoiceOfDefaultAtomicBasisSet
```

If the chosen basis set is not available in the pseudopotential file the nearest available one is used, the first if the index is smaller than 1, the last if it is larger than the number of basis sets in the file.

The atomic orbitals are used in the AO, AOJC and AOJCPW options of `TypeOfScfDiag`, and for the starting wave-functions of a PW calculation. A smaller basis set is faster but less accurate.

4.9 Saving wave-functions to disk

The calculations are more efficiently done keeping the wave-functions in main memory (RAM). However if the main memory is too small for the problem, either cache trashing occurs, impacting severely the computing efficiency, or the program aborts. Saving wave-functions to disk may allow to run the program with just a small penalty, if a modern disk (SSD, M.2 preferentially) is used. Useless if the number of k-points is just 1. Works better for large number of k-points. Before terminating the calculation the file is deleted.

- **TapeToSavePsi** (integer)

  Tape (I used them!) number for saving the wave-functions. If it is smaller than 10, wave-functions will be kept in main memory.

  Default value: `0`

- **PathToSavePsi** (string)

  Path to save the wave-functions.

  Default value: (empty)

  By default the `tmp_psi.dat` will be written in the current directory. If PathToSavePsi is `tmp` then the wave-functions will be saved to `tmp/tmp_psi.dat`.

4.10 Unfolding

If the structure is a supercell of some lattice it may be useful to plot the band structure in the "unfolded" Brillouin zone of the parent structure. The unfolding procedure was developed within an industrial collaboration (rede project), it was used in semiconductors with a parent fcc lattice, so unfolding was only extensively tested in that lattice.

- **Rede.Superlattice** (block)

  The block has three lines with three integers. Each line indicates how each of the supercell lattice vectors relate to the lattice vectors of the parent structure.

  Taking as an example the supercell described in `Structures/Supercells/cpw_Si6Ge6_Si(001).in`, which has 6 Si atomic layers and 6 germanium atomic layers on a (001) surface, the block is

```
%block Rede.Superlattice
          -1     1     0
```

```
            0     0     1
            3     3    -3
%endblock Rede.Superlattice
```

the underlying parent fcc structure has lattice vectors $\vec{A}_1 = (0, a/2, a/2)$, $\vec{A}_2 = (a/2, 0, a/2)$ and $\vec{A}_3 = (a/2, a/2, 0)$. The supercell vectors are $\vec{a}_1 = -\vec{A}_1 + \vec{A}_2 = (-a/2, a/2, 0)$, $\vec{a}_2 = \vec{A}_3 = (a/2, a/2, 0)$ and $\vec{a}_3 = 3\vec{A}_1 + 3\vec{A}_2 - 3\vec{A}_3 = (0, 0, 3a)$. Notice that the in-plane vectors have been rotated by 45°, their length is $\sqrt{2}a$ as the lattice constant is that of Si, it is consistent with epitaxy on Si (001). The third vector has a length slightly larger than $3a$ because it was allowed to relax in the presence of the larger Ge atoms.

- Rede.NumberOfLatticePlanes

  In a supercell grown on a surface indicates the number of lattice planes. It is used to help in plots to determine band alignments. In the above example you have six lattice planes (but twelve atomic planes). It must be present for unfolding to work. That is an "hack" to deal with back compatibility of some files, but do not influence the unfolding. This "requirement" should be removed in future releases.

5 Default output

The code writes to the default output, which can be redirected to a file. Depending on the choice of **PrintingLevel** the output will be different. We will use as a first example the file `Validation/Output/pw_ref_01.out`.

5.1 Initial information

On the top of the file we have the information about the version of code, and the date and time it was run.

```
density-functional pseudopotential plane-wave program version 4.99
run on the 19-Mar-21 at 13:30:54
```

notice that care was taken to code back the year 2000 bug in the 1990s.

Continuing with that example we find next some information about how many chemical species, and maximum number of atoms per chemical species the code is using.

```
The values set by size_mxdtyp_mxdatm_esdf are:

The value of mxdtyp is:     2
The value of mxdatm is:        8

input read from cpw.in
```

notice that the information about which subroutine printed the information is included in this case.

Next information are the lattice vectors,

```
 Primitive Translation Vectors
                      in a.u.                              in lattice units
a1=  0.125697E+02  0.000000E+00  0.000000E+00         1.225    0.000    0.000
a2= -0.418990E+01  0.118508E+02  0.000000E+00        -0.408    1.155    0.000
a3=  0.000000E+00  0.000000E+00  0.725711E+01         0.000    0.000    0.707
```

preceded by a warning that they were changed,

```
 WARNING:   avec(  3,  3)
 changed from       0.707106780000 to       0.707106781187
```

informing that the code recognized a value very close to $\sqrt{2}/2$ and changed it.
The line

```
     Single geometry calculation
```

informs that **MD.TypeOfRun** was ONE.
To be reproducible the information needed to generate the pseudopotentials is printed

```
  potentials :
  ------------

  Si  ca  nrl  nc
  19-SEP-18 atom 5.804  Improved Troullier - Martinskb-loc= 2
  3s(  2.00)  rc= 1.993p(  2.00)  rc= 1.993d(  0.00)  rc= 1.99
  nql=4000 delql=   0.0100
```

see the user guide of the pseudopotential generation code for details.
Next the other chosen or default parameters are printed

```
   Local Density Approximation (LDA) using
Ceperley and Alder as parametrized by Perdew and Zunger

dual aproximation is used

   The energy cutoff for wave-function kinetic energy is      10.000 Hartree

   SCF is converged when the difference in potentials is less then     0.00001000
   Iterative diagonalizationis converged when the error in |h psi - e psi| is less then     0

   The temperature for electron Fermi distribution is       0.000Kelvin

    Plane-wave basis calculation
```

see previous section for the keywords associated.

5.2 Crystal data

In the output the initial crystal data is printed,

```
     CRYSTAL DATA


    1081.02690815      volume

 real-space metric

      157.99703193     -52.66567731       0.00000000      metric  g11,g12,g13
                       157.99703193       0.00000000      metric      g22,g23
                                         52.66567731      metric          g33

   12.56968703      12.56968703       7.25711219   length 1,2,3  (a.u.)

  109.47122063      90.00000000      90.00000000   angle 12,13,23 (degrees)

      position (lattice coord.)         position (cartesian coord. a.u.)     no. type

    0.50000    0.50000    0.00000     0.72571E+01  0.00000E+00  0.00000E+00   1   Si    posi
   -0.12500    0.62500    0.00000     0.36286E+01  0.76973E+01  0.00000E+00   2   Si    posi
    0.00000    0.00000    0.00000     0.00000E+00  0.00000E+00  0.00000E+00   3   Si    posi
    0.37500    0.12500    0.00000     0.36286E+01 -0.25658E+01  0.00000E+00   4   Si    posi
    0.00000    0.50000    0.50000     0.36286E+01  0.51316E+01  0.36286E+01   5   Si    posi
    0.50000    0.00000    0.50000     0.36286E+01 -0.51316E+01  0.36286E+01   6   Ge    posi
    0.37500    0.62500    0.50000     0.72571E+01  0.25658E+01  0.36286E+01   7   Ge    posi
   -0.12500    0.12500    0.50000     0.00000E+00  0.25658E+01  0.36286E+01   8   Ge    posi
```

in what should be self-explanatory.

Here we encounter a characteristic of the output, in that we have a value `1081.02690815` followed by a "key" `volume`. This allows a quick search for that key in the output. For example

```
cpw2000-5.x.y/WORK$ grep volume ../Validation/Output/pw_ref_05.out
```

will return the volumes encoutered in Langevin a molecular dynamics with variational cell shape

```
    843.03621271      volume
    844.20060174      volume
    847.56015618      volume
    852.95790991      volume
    860.21678359      volume
    869.14551064      volume
    879.54271598      volume
    891.20293979      volume
      ...
   1092.59744351      volume
   1091.08256914      volume
   1089.72581021      volume
   1088.52562153      volume
   1087.47977919      volume
   1086.58556009      volume
   1085.84052430      volume
   1085.24216798      volume
```

in a form that would be trivial to plot (for example with `gnuplot`).

5.3 Crystal symmetry and reciprocal space info

The code recognizes the space group operations, although does not find its conventional name (we will later describe how to find it), it lists the symmetry operations, in lattice coordinates, first the matrix and than the eventual fractional coordinate.

```
    rotation matrices and fractional translations in lattice coordinates

    1     1  0  0     0  1  0     0  0  1         0.0000000000     0.0000000000     0.00000000
    2     0 -1  0    -1  0  0     0  0 -1        -0.5000000000    -0.5000000000     0.00000000
    3     0 -1  0    -1  0  0     0  0  1        -0.5000000000    -0.5000000000     0.00000000
    4     1  0  0     0  1  0     0  0 -1         0.0000000000     0.0000000000     0.00000000
```

we have 4 symmetry operations, the first is the identity and the last is the reflection with respect to the "z" axis. Remember that the code only uses the metric, and therefore the z-axis, which in this case is "obvious" because angles 13 and 23 are 90 degrees. In the general case it may be more complicated.

Next the output informs that there are 13117 G-vectors, but only 1868 are unrelated by spatial or temporal symmetry, and some information about the maximum values of g_i encountered in the expansion $\vec{G} = \sum_{i=1,3} g_i \vec{b}_i$ for the potential and charge densities.

```
    13117 G-vectors are set up in    1868 stars,  kmax =   17  17  10
```

It also informs that of the $4 \times 4 \times 4$ integration grid in reciprocal space, there are 12 unrelated by symmetry, and that it will calculate 24 orbitals

```
    12 k-points generated by program from parameters :
    n =    4    4    4      s =  0.50  0.50  0.50      nb =   24


  Computing time for starting (s):           0.01
```

5.4 The SCF cycle

The first information about the SCF cycle is the FFT grid size and the maximum and minimum values of the local self-consistent potential.

```
      in fft for local potential n =   24   24   16

  max and min of potential (Hartree)    0.1915   -4.8485    0.0000
```

In the first iterations one may encounter warnings of the type

```
   WARNING       WARNING:   After h_kb_dia
   The estimated error in energy has an accuracy
  of      7.3 digits
```

First, 7 digit accuracy is already quite good. To be fast the code does not try to be very accurate in the early SCF iterations, saving computing time. As long as the warnings do not persist into the final iteration, these warnings can be ignored, they are here because they can be helpful if things go wrong...

The information for iteration 8 is

```
  in fft for local potential n =   24   24   16

  max and min of potential (Hartree)    0.1819   -4.7630    0.0000

  the fermi level is at     5.9538 [eV]

 total energy =   -31.8310932988
  computing time for iteration     8          1.890
```

again `grep` may be used to find out what is happening along the SCF iterations.

5.5 Potential, energies and forces

Once self-consistency is achieved, the information about the reciprocal space expansion of the density and potential are printed,

```
   iteration number  9


    i   k-prot         Ek       den     V(out)    V(in)     delta V   Vionic

    1    0  0  0     0.00000  32.00000  -0.33659
    2   -1  0  0     0.14055   0.01194   0.00052   0.00051   0.00001  -0.01183
                              0.00024   0.00001   0.00001  -0.00000  -0.00000
    3   -1  1  0     0.18740   0.01175   0.00039   0.00039   0.00000  -0.01138
                              0.02455   0.00082   0.00081   0.00001  -0.02276
    4   -1 -1  0     0.37480   0.00032  -0.00005  -0.00005  -0.00000   0.00971
                              0.00000   0.00000   0.00000  -0.00000  -0.00000
    5    0  0  1     0.37480   0.00298   0.00018   0.00018   0.00000  -0.02912
                              0.00000   0.00000  -0.00000   0.00000   0.00000
    6   -2  1  0     0.51535   0.01475   0.00007   0.00007   0.00000   0.00858
                              0.00565   0.00006   0.00005   0.00000   0.00000
    7    0 -1 -1     0.51535  -0.01402  -0.00007  -0.00007  -0.00000  -0.00858
                             -0.00344  -0.00003  -0.00003  -0.00000   0.00000
    8   -2  0  0     0.56220   4.98977   0.03271   0.03271  -0.00000  -0.16014
                             -5.03324  -0.03301  -0.03301  -0.00000   0.15190
    9   -1  1  1     0.56220   4.98124   0.03270   0.03270  -0.00000  -0.16014
                              5.02352   0.03299   0.03299   0.00000  -0.15190
```

after the index, one has the expansion of the G-vector, the kinetic energy associated (half the length squared) the Fourier coefficient of the density, the Fourier coefficients of the input and output XC potential, followed by the difference `delta V` and the coefficients of the ionic pseudopotential. The self-consistency is done by potential mixing (many codes do density mixing) and the self-consistency condition is on `delta V`.

Next we have the details of the total energy

```
Iteration number  9    Energies (Ha)          Changes
--------------------

Alpha Term        =         0.917257
Kinetic  Energy   =        11.949083         0.000022
Local PP Energy   =        -8.515343        -0.000104
Nonlocal Energy   =         5.722961         0.000071
------------------------------------
Harris-Foulkes    =       -31.831093        -0.000000
------------------------------------
Eigenvalue Sum    =         1.020901        -0.000013
HXC  Correction   =         8.135799         0.000001
Hartree  Energy   =         2.166313         0.000017
XC       Energy   =        -9.562418        -0.000005
Ewald    Energy   =       -33.591688
------------------------------------
Total    Energy   =       -31.831093         0.000000
```

The last column has the change with respect to the previous iteration, it is not zero in the contributions, but has six decimals in the variational total energy, meaning nice convergence. it should be noticed that the Harris-Foulkes energy is identical (within the six decimals) with the total energy, another sign of good convergence.

Th total energy is the sum of the five last contributions, the Harris-Foulkes energy is the sum of the four first contributions plus the Ewald energy.

The stress tensor is given in both lattice and cartesian coordinates

```
Contravariant stress tensor  (a.u.)              Cartesian stress (GPa)
 0.000642    0.000195    0.000000         0.239973E+01    0.929093E-12    0.267779E-13
 0.000195    0.000642    0.000000         0.914063E-12    0.256751E+01   -0.116381E-13
 0.000000    0.000000    0.000836         0.254685E-13   -0.104815E-13    0.119894E+01

  0.00006986          2.05539463      pressure (au and GPa)                Total
```

its trace (divided by three) is the pressure.

Final information is the force on the atoms,

```
     Force (Lattice coord.)            Force (Cartesian coord. a.u)       no. type

 0.00053   -0.00062    0.00000    -0.69115E-03 -0.11838E-01  0.74502E-17   1   Si    forc
-0.00001    0.00001   -0.00000    -0.80366E-16  0.15823E-03 -0.33131E-15   2   Si    forc
 0.00062   -0.00053   -0.00000     0.69115E-03 -0.11838E-01 -0.30332E-15   3   Si    forc
-0.00058    0.00058    0.00000    -0.57519E-15  0.11843E-01  0.15964E-15   4   Si    forc
-0.00053    0.00053    0.00000    -0.14862E-15  0.10863E-01  0.88501E-16   5   Si    forc
-0.00063    0.00063   -0.00000    -0.51521E-15  0.12831E-01 -0.18928E-15   6   Ge    forc
-0.00032   -0.00091   -0.00000    -0.89184E-02 -0.60653E-02 -0.27596E-15   7   Ge    forc
 0.00091    0.00032   -0.00000     0.89184E-02 -0.60653E-02 -0.51362E-16   8   Ge    forc
```

indicating that we are close but not at equilibrium.

Computing time is given at the very end

```
  Total computing time (s):        47.74    Elapsed time (s):        12.01
```

with the ratio between computing and elapsed time indicating that the system had four cores.

6 Other files from cpw.exe

By design the code tries to write the minimum number of files. With grep one should be able to produce relevant information.

For example

```
cpw2000-5.x.y/WORK$ grep "4 Si position" ../Validation/Output/pw_ref_04.out
```

will produce the trajectory of the 4th atom in the simulation cell

```
      0.37456    0.12804    0.00365     0.36474E+01 -0.25300E+01  0.26210E-01   4   Si    posi
      0.37435    0.13059    0.01401     0.36644E+01 -0.25017E+01  0.10068E+00   4   Si    posi
      0.37460    0.13249    0.02300     0.36800E+01 -0.24848E+01  0.16520E+00   4   Si    posi
      0.37504    0.13449    0.02959     0.36977E+01 -0.24688E+01  0.21256E+00   4   Si    posi
      0.37525    0.13653    0.03398     0.37141E+01 -0.24500E+01  0.24414E+00   4   Si    posi
      0.37561    0.13857    0.03628     0.37314E+01 -0.24328E+01  0.26066E+00   4   Si    posi
      ....
      0.37419    0.12132   -0.02379     0.35960E+01 -0.25952E+01 -0.17094E+00   4   Si    posi
      0.37595    0.11977   -0.02458     0.35975E+01 -0.26292E+01 -0.17655E+00   4   Si    posi
      0.37730    0.11958   -0.02441     0.36060E+01 -0.26451E+01 -0.17536E+00   4   Si    posi
      0.37842    0.11900   -0.02299     0.36098E+01 -0.26624E+01 -0.16519E+00   4   Si    posi
      0.37909    0.11856   -0.02009     0.36116E+01 -0.26738E+01 -0.14429E+00   4   Si    posi
      0.37946    0.11954   -0.01604     0.36213E+01 -0.26676E+01 -0.11524E+00   4   Si    posi
```

allowing the user to plot it.

If the atoms move a `cpw.out` file will be written. With eventual modifications it can be used to continue the calculations with the final geometry. Notice that the default choices are different.

If the atoms move a `RESTART.DAT` file will be present. It may be used to continue the simulation for more steps, or restart if the calculation was interrupted by a power failure.

The most important file will be `PW_RHO_V.DAT`. It is a binary file, so it cannot be modified by accident but that may not allow portability between different operating systems. It contains the information about the crystal structure, charge density and potential.

6.1 The `PW_RHO_V.DAT` file

The file is written at the end of the self-consistent calculation by the subroutine `pw_rho_v_out` and read by the post-processing code with `pw_rho_v_in` (and `pw_rho_v_in_size` for the array dimensions). It is a Fortran unformatted sequential file. Files written with `gfortran` or `ifx` on the usual (little-endian) computers can be exchanged between them, but files from computers with a different byte order or record markers cannot be read directly. It is not supposed to be an *archival* file, although there is an effort to assure back-compatibility.

The file is self contained. Besides the self-consistent charge density and potential it has the crystal structure, the symmetry operations, the description of the plane-wave calculation and

the pseudopotentials (with the atomic orbitals) that were used, so the post-processing does not need the `cpw.in` or the pseudopotential files.

Content of the file

The records of the present version (2026.1) of the file are listed below. Character variables have the indicated length, integers are default integers and real and complex numbers are double precision (64 bits).

record	content	comments
1	`ntype, ng, ns, mxdl, ntrans, mxdlao, mxdset`	numbers of atom types, G-vectors, stars, symmetry operations, and array dimensions for the pseudopotentials
2	`natom(1:ntype)`	number of atoms of each type
3	`bdate, btime`	date (9 characters) and time (8 characters)
4	`author, flgscf, flgdal`	exchange-correlation (4), SCF flag (6), dual approximation (4)
5	`emax, teleck, nx, ny, nz, sx, sy, sz, nband, alatt, efermi`	cutoff (Hartree), electronic temperature (K), Monkhorst-Pack grid and shift, number of bands, lattice constant, Fermi level (Hartree)
6	`meta_pwdat, meta_cpw2000`	metadata of the input file and of the calculation (250 each)
7	`adot, mtrx, tnp`	metric (bohr^2) and the `ntrans` symmetry operations
8	`nameat(1:ntype)`	chemical symbols (2 characters)
9	`rat(3,natom(i))`	one record per atom type, lattice coordinates
	`kmax(3)`	maximum components of the G-vectors
	`kgv(3,ng)`	G-vectors in reciprocal lattice coordinates, ordered by stars
	`phase(i), conj(i), i=1,ng`	phase and conjugation flag of each G-vector in its star
	`mstar(ns)`	number of G-vectors in each star
	`den(ns)`	valence charge density
	`den(ns)-dens(ns)`	bonding charge density (minus superposition of atoms)
	`veff(ns)`	effective potential (local+Hartree+exchange-correlation, Hartree)
	`vtau(ns)`	$\partial(\rho\epsilon_{xc})/\partial\tau$, only for meta-GGA with τ from the wave-functions (generalized Kohn-Sham)
	pseudopotentials	for each atom type, see below

The densities and potentials are given by their Fourier coefficients for the first (prototype) G-vector of each symmetry star. The coefficient for the `i`-th G-vector is the coefficient of its star times `phase(i)`, and its complex conjugate is taken if `conj(i)` is -1. The density is normalized to the cell, `den(1)` is the number of valence electrons in the unit cell.

For each atom type the pseudopotential is copied by default from the `Xy_POTKB_F.DAT` file (generated by the `atom` code), where `Xy` is the chemical symbol, keeping its units (Rydberg), the conversion to Hartree is done on reading:

- the header with the chemical symbol, correlation, relativistic and core-correction flags and title;
- `izv, nql, delql, vql0`, the valence, number and step of the points in reciprocal space;
- the number of non-local projectors, their angular momenta, normalizations and reference energies (with separate values for $j = l \pm 1/2$ in relativistic pseudopotentials);
- the local potential, the Kleinman-Bylander projectors, the partial core charge and the atomic valence charge, one value per record;
- `nqwf, delqwf, norbat, n_bsets` followed, for each of the `n_bsets` atomic basis sets, by the number of orbitals and for each orbital its angular momentum, energy and Fourier transform (one value per record);
- the partial core kinetic energy density, in a single record (zero if absent).

For details read the user guide of the `atom` code. See section on pseudopotential files for other possible pseudopotential file names.

Versions of the file

The format of the file evolved as the code was developed. Older files may still be read, the version is identified from the length of some records and printed by the post-processing code, for example

```
The version of the file format is 2021.1
```

version	date	changes
2013.0	December 2013	first record `ntype, ng, ns, mxdl`, record 6 was `pwline` (60 characters)
2013.1	August 2014	record 6 with `pwline`, title and subtitle (140 characters)
2013.2	January 2017	record 6 with `meta_pwdat, meta_cpw2000`
2021.0	September 2021	`ntrans, mxdlao` in record 1, symmetry operations in record 7
2021.1	November 2021	`efermi` in record 5
2026.0	2026	`mxdset`, several atomic basis sets (development only)
2026.1	October 2026	`vtau` and partial core kinetic energy density

When reading older files the missing quantities are given default values: before 2021.0 the symmetry operations are not in the file and `mxdlao` is taken as 4; before 2021.1 the Fermi level is not known and is estimated by the post-processing; before 2026.1 there is a single atomic basis set, the partial core kinetic energy density is zero, and for a meta-GGA with τ from the wave-functions `vtau` is set to zero with a warning, so the self-consistent calculation should be repeated. Files of the 2026.0 development version are not recognized.

7 Post processing

Once the self-consistent potential for some structure has been saved to the `PW_RHO_V.DAT` file, one can proceed to calculate many properties witn a post-processing code,

```
cpw2000-5.x.y/WORK$ ../cpw_post_process.exe
```

it is mainly an interactive code, asking questions and proceeding according to the answers. All the answers are printed back to a file `replay_post.dat`. This way the user will have a record of the answers, and more important, may rerun the post processing, with eventually slight changes.

```
cpw2000-5.x.y/WORK$ cp replay_post.dat replay.dat
```

and then, with optional minor editing of the file,

```
cpw2000-5.x.y/WORK$ ../cpw_post_process.exe < replay.dat
```

The post processing code is in fact several independent subroutines (sub-programs) called from the same top level program.

That program loops at most 100 times (just edit if you want more!) asking what the user wants and reads an integer entered by the user.

If the integer is zero, the program stops. At lower levels of the program a choice of zero would return the execution to the calling level.

If the integer is not in the allowed range, the question is repeated. In lower levels of the program an out of range choice gives the user a second chance. If the user gives again a wrong choice the code returns to the calling level.

Looking at the examples

7.1 Band structures, density of states, optical response, quantum geometric,...

This is the part of the post-processing that has the most options. The code gives a summary of the main parameters of the SCF calculation (geometry, pseudopotentials, etc,...) and then asks if the user wants the default values for the plane-wave cutoff, eigenvalue precision, Fermi energy estimate, atomic basis sets, and the use of the dual approximation. Answering "yes" is safe and is the right choice in most cases. The code will then use the same cutoff as the SCF calculation, the dual approximation, an eigenvalue precision of 0.0005 Hartree, the estimate of the Fermi energy from the SCF calculation, and the first atomic basis set of each atom.

If the answer is "no" the code asks about each parameter in turn.

- Plane-wave cutoff

  The code informs the user what cutoff was used for the plane-wave expansion in the SCF calculation and asks the user what cutoff (s)he wants to use. Choosing the same cutoff is a safe option. Only Hartree units are allowed.

- Dual approximation

  The code asks if it can use the dual approximation, again answering "yes" is safe. The choice can be changed later with the option "Reset dual approximation flag".

- Eigenvalue precision

  You can change the eigenvalue precision of the iterative diagonalization if you are not confortable with the default. You can manage with a low precision in most cases. If you want to change it the code asks for the number of decimals (in eV) of the precision, a value between 3 and 8.

- Fermi energy

The code will read from the file a reasonable guess of the Fermi energy. In several cases it will shift the calculated eigenvalues by that value. The estimate is not precise, and if you have a better estimate, for example from a band structure or density of states, you can enter it (in eV). The code warns you if the new value differs by more than 2 eV from the estimate. It only affects the shift of the eigenvalues, all other quantities are unaffected.

- Atomic basis set

  This question is only asked if the pseudopotential file of some atom has several atomic basis sets. The code asks if you want to use other than the first basis set. In the affirmative case, for each atom with several basis sets, it lists the number of orbitals and their angular momentum (`s`, `p`, `d`, `f`) in each basis set, and asks which one to use. If the chosen basis set does not exist the nearest available one is used. The choice made in the SCF calculation with `ChoiceOfDefaultAtomicBasisSet` is not stored in the `PW_RHO_V.DAT` file, so the default is always the first basis set. The atomic basis set is used by the diagonalizations in atomic orbitals, in the starting guess of the iterative diagonalization, and when the results are projected on atomic orbitals.

The code then gives a choice of several types of calculations.

7.1.1 Band structure

The first option is to plot a band structure. For that you need a path in the Brillouin zone. That information must in a file called `BAND_LINES.DAT` in the working directory. In the `Doc` folder the user can find information from QUANTUM ESPRESSO and ELK that can help write or modify the `BAND_LINES.DAT` file. In `Structures/Band_Lines` the user can find some examples.

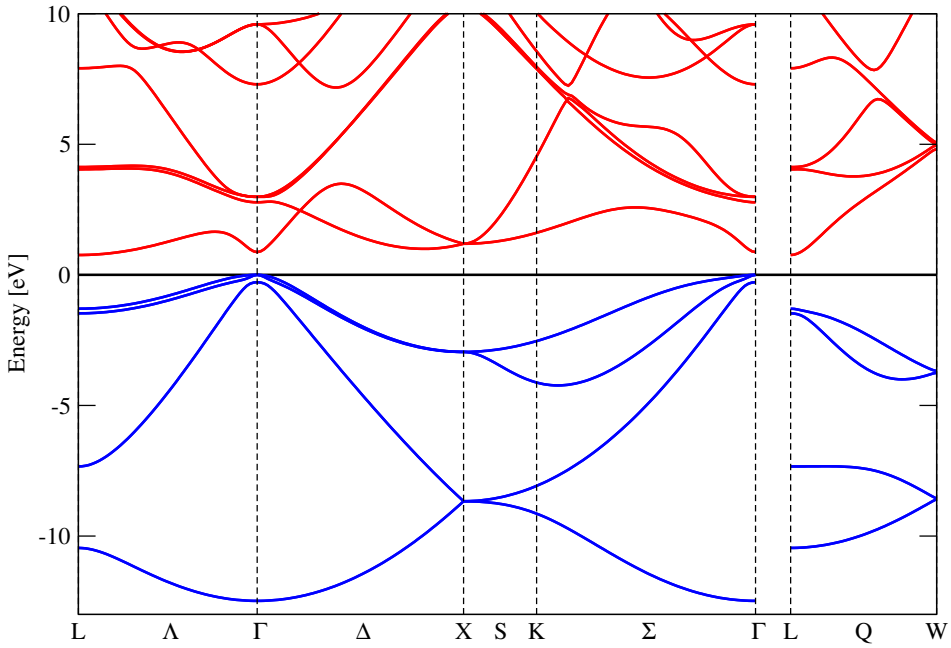
An online help to generate a BZ path can be found in `https://seekpath.materialscloud.io/`, based on Hinuma et. al., Comp. Mat. Sci. **128**, 140 (2017). Go to the Crystallography section of this document to find how to get a `.cif` or QUANTUM ESPRESSO input file that can be uploaded to that site. The `VASP KPOINTS input for LDA/GGA` result can be easily copied/pasted/edited to the texttttBAND_LINES.DAT format.

An example of a `BAND_LINES.DAT` file for an fcc lattice is

```
5       11          0.01
0.5     0.5     0.5         0.0     0.0     0.0     80      L   Lambda   Gamma
0.0     0.0     0.0         0.0     0.5     0.5     80      Gamma  Delta  X
0.0     0.5     0.5         0.25    0.625   0.625   20      X       S     K
0.25    0.625   0.625       1.0     1.0     1.0     60      K    Sigma   Gamma
0.5     0.5     0.5         0.25    0.75    0.5     60      L      Q     W
```

In the first line one must specify the number of lines, 5 in this case, and how many bands (without spin) one wants to plot. A third optional value gives the spacing between calcualted points.

Each subsequent line indicates a pannel in the band structure plot. The initial and final point are indicated in reciprocal lattice coordinates. In the example the conventional choice of primitive lattice vectors was used. After the coordinates of the final point one has the number of points in that pannel, it is **not** used if in the first line the spacing between points is specified. Finally the labels of the first point, line and last point are specified. The code understands the meaning of the greek letters `Gamma, Lambda, Delta, Sigma`.



A band structure for Ge is shown in the figure. Notice that if the end of one pannel is the beginning of the next the pannels are joined. If not a small space is introduced.

The band structure can be calculted by several methods.

- Full plane wave basis diagonalization

  This is the "normal" case, slower but most precise. You will produced files with and without spin-orbit, in the GRACE and `gnuplot` formats.

  The higher quality is the GRACE file, `band_so.agr` and `band.agr` for the case with and without spin-orbit. But you have to have installed the graphical software, `https://plasma-gate.weizmann.ac.il/Grace/`, it is available on most linux distributions. However development seems to have stopped. It is a good start for publication quality figures.

  Available in all distributions is gnuplot, just type

  ```
  cpw2000-5.x.y/WORK$ gnuplot band_so.gp
  ```

  and you will have a window with the band structure.

- Diagonalization in a Luttinger-Kohn basis

  This is very fast but only accurate around the Γ point of the Brillouin zone. It will produce the similar files as the previous case but with an added `_lk` in the file name.

- k.p method

  It is a second order expansion of the hamiltonian in a Luttinger-Kohn basis around the Γ point. So it is very similar to the previous method, but now he files have a `_kp` added to the file name.

- 2-k-point Luttinger-Kohn interpolation

  This is a very good compromise between speed and accuracy, but needs some additional information. It calculates the bands on a few points and then interpolates with the GLK method between them.

The code first asks how many interpolation points you want between calculated points. Notice that the values in `BAND_LINES.DAT` that control the number of k-points is *after* interpolation. Besides the starting and end points on each pannel, you need to calculate a few (1 to 3) intermediate points to have a good interpolation, so the answer to that quaetion depends on how many final points you are going to have. The speedup in computing time is almost the number of intermediate points. Accuracy and computing time are again the compromises you have to take.

The code than asks if you want to accept the defaults of the calculation. If unsure accept the defaults. Otherwise read the code and the relevant article.

This option only produces the `grace` figures. They have an added `_lk_int` to the file names. There is also a `band_lk_ref.agr` file with the reference (fully calculated) bands. It should have visible straight lines if you are using the method efficiently.

- Band-unfolding with full diagonalization

  For a supercell, the band structure is plotted in the Brillouin zone of the parent structure, with the weight of each state in the unfolded band. The supercell is given by the `Rede.Superlattice` block (see the Unfolding section of the `cpw.in` file), and `BAND_LINES.DAT` uses the reciprocal lattice coordinates of the parent structure. If that information is absent the band structure is not unfolded. If the pseudopotential files have atomic orbitals, the code asks if the hamiltonian should be diagonalized with plane-waves, in the atomic orbitals followed by a single iteration Jacobi correction, or only in the atomic orbitals. The files are `band.agr` and `band_so.agr`.

  The code first asks "Are you a human?" (a nod to the movie Blade Runner). A casual user answers yes. The answer no is for a calculation split in several independent jobs (workers) that restart from the partial results saved in `band_fold_rec.dat`. That job flow needs some previous setup.

- Prepare file for later ploting with information about atomic orbitals with unfolding

  It is similar to the previous option, but it also calculates the projections of the states on the atomic orbitals, and writes them, with the unfolding weights, in the files `BAND.DAT` and `BAND_SO.DAT`. Those files are used by the graphical interfaces described in the Toolbox section. The reference plots `band_fld_ref.agr` and `band_fld_ref_so.agr` are also written. It needs atomic orbitals in the pseudopotential files.

- Band-unfolding with 2-k-point Luttinger-Kohn interpolation

  It combines the unfolding with the interpolation of the fourth option. The code asks if you want the default values of the interpolation, if unsure accept them. The files are `band.agr` and `band_so.agr`.

- Diagonalization in the atomic orbital basis

  The hamiltonian is diagonalized in the atomic orbitals, either alone, or followed by a single iteration Jacobi correction. The first is fast but only qualitative, the second is close to the full plane-wave calculation. The atomic basis set is the one chosen in the questions about the default parameters, and the number of bands cannot exceed the number of atomic orbitals in the unit cell. The files have `_ao` or `_aojc` added to the name, for example `band_ao_so.agr`.

7.1.2 Prepare file for density of states

This option writes files that allow later ploting of the density of states. It needs a file `DOS_MESH.DAT` to be present in the working directory, and it will write a file `PW_DOS.DAT` with the information.

An example of a `DOS_MESH.DAT` file is

```
12  8 8 8   0.0 0.0 0.0
```

it has a single line, the first integer is the desired number of bands, the next three integers are the mesh in reciprocal space, and the last three are the shift of the mesh. The section about Brillouin zone sampling explains their meaning.

7.1.3 Properties for single k-vector

This option allows to analyze in detail the eigenstates for a single k-vector. The code asks how many bands the user wants to analyze, followed by the lattice coordinates of the k-vector.

7.1.4 Prepare file for dielectric function calculation

This is similar to the prepare file for density of states. Follow the instructions on that section.

7.1.5 Calculate effective band masses

This section allows the calculation of effective band masses with three different methods.

- Topological tensor

  This is the most accurate method. It will ask first how many bands (not counting spin) should be calculated. Afterwards the code asks which coordinate system the user wants to use, and then what are the coordinates of the desired k-point.

  The code will ask if the results should include spin-orbit, and, in the affirmative case, which method should be used to diagonalize the Hamiltonian matrix.

  - Full diagonalization

    Recommended if you can afford a full diagonalize a matrix of the indicated size, as it is the most accurate.
  - Iterative diagonalization

    Much faster than full diagonalization for large matrices, but may be subject to noise.
  - First order perturbation in spin-orbit

    Uses perturbation theory instead of matrix diagonalization.

  The code will provide a list of energy levels, and then ask if the user wants masses in a particular direction. If the user answers 'yes' it will ask what coordinate system the user wants to use and then enter a loop where the user can enter directions in the chosen coordinate system, and the code will print the effective masses in that direction for all bands.

Once the user indicates that it is done with directions, the code will ask if the user wants a figure of the masses for one band in all directions. If the user answers 'yes' the code will ask which band the user wants, and writes a file `band_`n`[_so].dat`, where n is the band number, with the relevant information. The instructions to produce a figure with Mathematica are printed out.

- k.p method

  This is a fast method, but in the end it will use a finite difference. Read in the next section the dangers of finite differences calculations of masses. It will ask first how many bands (not counting spin) should be included in the exploratory calculation. It is not the number of bands of the k.p model. The code gives an idea of the recommended *order of magnitude* of the answer. Afterwards the code asks which coordinate system the user wants to use, and then what what are the coordinates of the desired k-point. It will then give the suggested values of the size of the k.p method. Choose one of the suggestions, unless you have a strong reason to use another value.

- Finite differences

  This is a tricky procedure in the case of band crossings (or near avoided band crossings). Use it with the utmost care.

7.1.6 Calculate quantum geometric quantities

This section of the code will calculate the Berry curvature, quantum metric, orbital magnetic moment and effective mass tensors. The analysis will proceed by *energy level* not by state.

The first question asked by the code is how many bands (not counting spin) should be examined. It gives as hint the number of occupied states if the system was an insulator. Enter a somewhat larger number. Afterwards the code asks which coordinate system the user wants to use, and then what what are the coordinates of the desired k-point. It will then ask if spin-orbit should be taken into account, and, in the affirmative case, which method should be used to diagonalize the Hamiltonian matrix. See previous section for the meaning of the choices.

The code will then print the *energy levels* with spin polarization for each state in that level. It will then enter a loop asking which level should be examined, and which quantity should be printed out. The options are

- All quantities

  All the options below

- Berry curvature

  An example of output for a non-degenerate level of hexagonal Te is

```
      Berry curvature tensor for level with energy         5.480


  primitive (reciprocal) lattice coordinates

    0.000    -189.681      -0.000
  189.681       0.000       0.000
    0.000      -0.000       0.000
```

```
 Cartesian lattice coordinates

    0.000    -292.927      -0.000                        berry_curv    1    1   18
  292.927       0.000       0.000                        berry_curv    1    2   18
    0.000      -0.000       0.000                        berry_curv    1    3   18


Trace over coordinates       0.000


 Associated pseudo-vector

         0.000       0.000    -292.927                    berry_curvvector   18

 Module:      292.927
```

The 3×3 tensor is given in lattice coordinates and cartesian coordinates. The columns and rows correspond to the x, y and z directions. The meaning of those directions are with respect to the conventional orientation indicated at the top of the SCF output. In this case it is what we "expect" from an hexagonal crystal. The tensor is antisymmetric, only $\Omega_{xy} = -\Omega_{yx}$ are non-zero. The antisymmetric tensor is converted to a pseudo-vector with only the Ω_z component. The trace is trivially zero. Notice that some quantities are followed by keywords so that they can be found by a `grep` command.

The output for a degenerate level is more complicated, here again an output for hexagonal Te, at another k-point, for a double degenerate energy level.

```
Berry curvature tensor for level with energy         4.464


 primitive (reciprocal) lattice coordinates

    0.000       6.017      -0.000          -0.000      -0.000       0.101
   -6.017       0.000      -0.000          -0.000      -0.000       0.201
    0.000       0.000       0.000          -0.101      -0.201      -0.000

    0.000       0.000       0.101           0.000      -6.017       0.000
    0.000       0.000       0.201           6.017       0.000       0.000
   -0.101      -0.201       0.000          -0.000      -0.000       0.000


 Cartesian lattice coordinates

   -0.000       9.292      -0.000          -0.000      -0.000      -0.000
   -9.292       0.000      -0.000           0.000      -0.000       0.413
    0.000       0.000       0.000          -0.000      -0.413      -0.000
```

```
   0.000      -0.000       0.000          0.000      -9.292       0.000
   0.000       0.000       0.413          9.292       0.000       0.000
   0.000      -0.413       0.000         -0.000      -0.000       0.000


 Trace over coordinates

  -0.000          -0.000
   0.000           0.000

 Trace over trace of coordinates      -0.000


 Trace over energy levels

       -0.000      -0.000      -0.000
        0.000       0.000      -0.000
        0.000       0.000       0.000


 Pseudo-vector associated with the trace.

       -0.000       0.000      -0.000                    berry_curvvector    9

 Module:        0.000
```

Now we have a 2×2 group of $3times3$ matrices. The rows and columns of the $3times3$ matrices are the directions x, y and z, the rows and columns of the 2×2 grouping are associated with each state within the doubly degenerate level. As care was taken to choose the two states that span the degenerate energy eigenspace, there is a nice apparent structure to this four index tensor.

- Quantum metric

  The quantum metric is a symmetric tensor. An example of output for a non-degenerate level of hexagonal Te is

```
 Quantum metric tensor for level with energy         5.480


 primitive (reciprocal) lattice coordinates

  244.291     122.146      -0.000
  122.146     244.291      -0.000
   -0.000      -0.000     163.410


 Cartesian lattice coordinates
```

```
  326.719      -0.000      -0.000                    qua_metric    1    1   18
   -0.000     326.719      -0.000                    qua_metric    1    2   18
   -0.000      -0.000     516.225                    qua_metric    1    3   18


 Trace over coordinates    1169.664


 Trace over trace of levels    1169.664


 Eigenvalues:      326.719     326.719     516.225
      0.551E+08   Determinant qua_metric   18
```

The structure of the results is similar to the case of Berry curvature, but as the tensor is symmetric the trace is non-zero. The code calculates a few of the invariants of the tensor.

An example for the degenerate case is

```
Quantum metric tensor for level with energy         5.793


  primitive (reciprocal) lattice coordinates

   238.414     119.207      -0.000         -33.231      33.231     -29.478
   119.207     238.414      -0.000          33.231      66.463      -5.677
    -0.000      -0.000      58.036          23.800      -5.677       0.000

   -33.231      33.231      23.800         238.414     119.207      -0.000
    33.231      66.463      -5.677         119.207     238.414      -0.000
   -29.478      -5.677       0.000          -0.000      -0.000      58.036


  Cartesian lattice coordinates

   318.858       0.000       0.000         -88.889       0.000     -63.227
     0.000     318.858      -0.000          -0.000      88.889     -11.670
     0.000      -0.000     183.341          63.227     -11.670       0.000

   -88.889      -0.000      63.227         318.858       0.000      -0.000
     0.000      88.889     -11.670           0.000     318.858      -0.000
   -63.227     -11.670       0.000          -0.000      -0.000     183.341


  Trace over coordinates

   821.058          -0.000
```

```
   -0.000         821.058

 Trace over trace of coordinates    1642.116


Eigenvalues:      821.058     821.058
     0.674E+06   Determinant qua_metric   13


 Trace over energy levels

       637.717       0.000      -0.000
         0.000     637.717      -0.000
        -0.000      -0.000     366.682

Trace over trace of levels    1642.116


Eigenvalues:      366.682     637.717     637.717
     0.149E+09   Determinant qua_metric   13
```

Besides the full tensor the code also calculate some traces and determinants.

- Orbital magnetic momentum

  The orbital magnetic momentum is antisymmetric like the Berry curvature. See that case for details.

- Effective mass tensor

  The effective mass tensor is symmetric like the quantum metric. See that case for details.

- Interband contribution to mass tensor

  The effective mass tensor has two contributions, the interband contribution is the real part of a complex tensor whose imaginary part is the orbital magnetic momentum. Here it is printed separately so the full complex tensor is available.

7.1.7 Reset dual approximation flag

This option allows to toggle the use of the dual approximation. Useful if you want to check its consequences.

7.2 Modified tight binding

For a fast dense sampling of the Brillouin zone, a modified tight-binding scheme can be used. It has many similarities with Wannier interpolation, and in fact can be used to write the files required to call WANNIER90.

The hamiltonian and overlap in the atomic orbital basis are calculated in a uniform grid of k-points and Fourier interpolated. If the pseudopotential files have several atomic basis sets the code asks which one to use. At the grid points the hamiltonian is modified so that its eigenvalues are those of the plane-wave calculation, and the code asks how it should be done.

- Projection of the plane-wave states (recommended)

  The plane-wave states are projected on the atomic orbitals, and those that are well represented (at least half of the state in the atomic orbital space) become, after a symmetric orthonormalization, the eigenvectors of the hamiltonian with the plane-wave energies. The rest of the atomic orbital space keeps the atomic orbital hamiltonian. Plane-wave states that cannot be described by the atomic orbitals, for example the interlayer states of layered materials, are left out.

- Matching of atomic orbital and plane-wave states

  The older method. Each atomic orbital state gets the energy of the plane-wave state with the largest overlap, if the overlap is large enough.

The grid must be dense enough for the Fourier interpolation of the atomic orbital hamiltonian. For hexagonal crystals use in plane a multiple of three, for example `6 6 4` for graphite instead of `4 4 4`.

7.3 Plot of charge densities and potentials

Plots or prepares the plots of real functions electron density, $\rho(\vec{r})$, effective potential, $v_{\text{eff}}(\vec{r})$ and some potential contributions.

It can generate a three dimensional plot in the `".xsf` format of XCrysDen compatible with VESTA, or a two dimensional level plot in a plane chosen by the user.

An interesting capability is to plot along a line these quantities averaged over the perpendicular plane. It is very useful to find band alignments.

7.4 Plot densities of states

The data for the calculation of the density of states and respective plot can be generated with different approaches. This section of post processing reads the file with the data and generates the plot.

Choices of graphics options are separated from the sometimes very heavy calculations of the bands throughout the Brillouin zone.

7.5 Exploration of a k.p model

The first option may store a k.p model in a file. This section allows the exploration of that model.

7.6 Plot dielectric function

Like the case of density of states, the data for the dielectric function is first calculated throughout the Brillouin zone, the choices of graphics options are chosen in this section of the code.

7.7 Plot of the wave-functions

It is similar to the plots of the charge density, but now the function to be plotted is complex, adding an extra choice of real part, imaginary part, or module, plus obviously the choice of

which state to plot. Both 2D and 3D options are available.

Before the plots the code asks the same questions about the default parameters (plane-wave cutoff, dual approximation, eigenvalue precision, atomic basis set) as the band structure section. As accuracy is not paramount for a plot, if the pseudopotential files have atomic orbitals, the code then offers the choice of diagonalizing the hamiltonian with the full plane-wave basis, in the atomic orbitals followed by a single iteration Jacobi correction, or only in the atomic orbitals. The last two are much faster. With atomic orbitals the number of bands cannot exceed the number of atomic orbitals in the unit cell, if more bands are requested the number is reduced accordingly. A smaller atomic basis set makes the calculation faster but less accurate.

7.8 Crystalography

This section allows a reveryough examination of the crystal structure.

7.9 Interface to Quantum Espresso

Quantum Espresso has many more developers than the present code. This section writes a QE input file. The pseudopotential code can write the required pseudopotentials, including Troullier-Martins pseudopotentials with spin-orbit (a feature that is not available in the QE toolchain). The consistency between the two codes are at the 4th or 5th decimal place in the total energy.

8 Where to find crystal structures

Many example of `cpw.in` files can be found in the `Structures` directory. Besides (almost) all the elements, there are many files collected over the years.

There is an auxiliary program in the toolbox, `Tools/gen_PW.f90` that can help the user to create a `cpw.in` file by answering some questions.

As mentioned before, the input file was chosen to be very similar to the SIESTA input file, so the user adapt it easily.

Another option is to convert any crystal file in the `".xyz` format which is sufficiently simple to be adapted.

9 Toolbox

In `Tools` directory the user can find a few useful tools.

Most are sufficiently short that the user can easily read and modify the code.

It also has programs that call one of the options of post-processing. They were used to develop those options.

9.1 Equation of state

The program `Tools/eqst/eqst.f90` fits the energy as a function of volume, $E(V)$, calculated for one or several crystal structures to the Murnaghan (F. D. Murnaghan, Proc. Natl. Acad.

Sci. **30**, 244 (1944)) or the Birch-Murnaghan (third order) (F. Birch, J. Geophys. Res. **83**, 1257 (1978)) equation of state. For each structure it gives the equilibrium volume V_0, the energy $E_0 = E(V_0)$, the bulk modulus B_0 and its pressure derivative B_0'. If several structures are given, it compares their enthalpies $H(p) = E + pV$, finds the transition pressures between them, and prepares plots of $E(V)$ and of the volume as a function of pressure, $V(p)$, of the most stable structure.

The two equations of state are

$$E(V) = E_0 + \frac{B_0 V}{B_0'}\left[\frac{(V_0/V)^{B_0'}}{B_0'-1}+1\right] - \frac{B_0 V_0}{B_0'-1} \qquad \text{(Murnaghan)} \tag{1}$$

and

$$E(V) = E_0 + \frac{9V_0B_0}{16}\left\{\left[(V_0/V)^{2/3}-1\right]^3 B_0' + \left[(V_0/V)^{2/3}-1\right]^2\left[6-4(V_0/V)^{2/3}\right]\right\} \qquad \text{(Birch)}. \tag{2}$$

The Birch fit is linear except for V_0, which is found by minimizing the mean square error. In the Murnaghan fit the non-linear parameter B_0' is searched in the interval $2 \le B_0' \le 7$. The results of both fits should be similar. If they are not, the calculated points should be checked (are they converged? do they cover a reasonable range of volumes around V_0?).

9.1.1 Compiling and running

The main program is in `Tools/eqst/eqst.f90`, and the subroutines it calls are in the `libpw` library. It must be compiled with the same compiler that was used for the library, for example with the link created in section 2.3

```
cpw2000-5.x.y/Tools/eqst$ gfortran -o eqst.exe eqst.f90 ../../libpw.a
```

The data are read from the file `eqst.dat` in the directory where the program is run. If that file does not exist, the program asks for the name of the data file, and stops if that file does not exist either.

```
cpw2000-5.x.y/WORK$ ../Tools/eqst/eqst.exe
```

or, to use a different data file without being asked,

```
cpw2000-5.x.y/WORK$ echo eqst_Si.dat | ../Tools/eqst/eqst.exe > eqst_Si.out
```

9.1.2 The `eqst.dat` file

The data file is read line by line, in free format. Text after the expected values in a line is ignored, so comments may be added at the end of a line, except on the line with the volume factor (see below), where it would be read as the label if the label is missing. The file has the following structure:

Line 1: type of equation of state, `BIRCH` or `MURNA` (all uppercase or all lowercase). With any other value the Murnaghan equation of state is used.

Line 2: number of structures, `nstr` (positive integer).

A block for each structure (see below).

Last line: only read if there is more than one structure. Maximum pressure `pmax` (in GPa) and number of points `npts` for the plots as a function of pressure. If `pmax` is not in the interval $[0, 5000]$ it is reset to 100 GPa, and if `npts` is not in the interval $[10, 5000]$ it is reset to 50.

Each block has the following structure:

First line of the block: number of calculated points, `npt`, for this structure (integer, at least 4).

Second line of the block: `volfac` [`nform`] [`label`], where the items in square brackets are optional.

- `volfac` (real) is the volume factor, which defines the meaning of the first number in the data lines,
  - `volfac` > 0: the data lines have the lattice constant a, and the volume of the cell is $V =$ `volfac` $\times a^3$. For example, `volfac` $= 0.25$ for the primitive cell of a face centered cubic lattice, 0.5 for body centered cubic and 1 for simple cubic.
  - `volfac` $= 0$: the data lines have the volume of the cell.
  - `volfac` < 0: epitaxial case, where only the dimension of the cell perpendicular to $\vec{a}_1$ and $\vec{a}_2$ changes. The data lines have the volume of the cell, and |`volfac`| is the area of the base of the cell (in bohr^2), so that the height of the cell is $V/$|`volfac`|. Instead of the bulk modulus the corresponding elastic constant is printed.
- `nform` (positive integer, default 1) is the number of formula units in the cell. It is only used when comparing different structures: the plots and the enthalpies used to find the transition pressures are per formula unit. The results printed for each structure are for the cell. For example, for silicon in the diamond structure with the primitive cell (2 atoms) and in the fcc structure (1 atom), `nform` should be 2 and 1. The second item of the line is taken as `nform` only if it is written only with digits, otherwise it is taken as the label. Older files, without `nform`, can therefore still be read.
- `label` (string with at most 10 characters and without blanks) identifies the structure in the output and plots. If it is missing a default label is created. As it is used as a title in `gnuplot`, an underscore should be written as `\_` (the backslash is removed in the `python` scripts). A label cannot be made only of digits.

The whole line must have less than 100 characters.

The next `npt` lines: lattice constant (bohr) or volume of the cell (bohr^3), depending on `volfac`, and the total energy of the cell (hartree). The points do not need to be ordered.

There is no limit on the number of structures or points, as the program reads the file twice, first to find the dimensions of the arrays.

As an example, the following file has the energies of silicon calculated in the diamond (2 atoms in the primitive cell), body centered cubic and face centered cubic structures

```
BIRCH
 3
 9
```

```
 0.25000000    2    diamond
 9.61978677   -7.92021896
 9.76120812   -7.92662746
 9.90262948   -7.93085811
10.04405083   -7.93317278
10.18547219   -7.93376582
10.32689355   -7.93287971
10.46831490   -7.93068538
10.60973626   -7.92736393
10.75115762   -7.92305672
 9
 0.50000000    1    bcc
 5.23927021   -3.93972397
 5.35474026   -3.94657964
 5.47021032   -3.95117850
 5.58568037   -3.95370854
 5.70115043   -3.95454963
 5.81662048   -3.95393709
 5.93209053   -3.95221012
 6.04756059   -3.94945947
 6.16303064   -3.94605662
 9
 0.25000000    1    fcc
 6.64376027   -3.93680372
 6.78518162   -3.94144018
 6.92660298   -3.94449813
 7.06802434   -3.94615157
 7.20944569   -3.94659005
 7.35086705   -3.94611966
 7.49228841   -3.94488143
 7.63370976   -3.94308661
 7.77513112   -3.94073548
200.000  101
```

9.1.3 Output

For each structure the program prints the input points, the scaled energies and their fitted values, a measure of the quality of the fit, and the results of the fit, for the silicon example in the diamond structure

```
aeq(a.u.)= 10.1690   Veq(a.u.)=  262.894   E0(Hartree)=  -7.93377
B0(GPa)=   96.88   B0PRIM=   4.16

Lattice constant in Angstroms:    5.38123
```

where the equilibrium volume and energy are for the cell of the data (2 formula units in this case). For the fit the energies are internally converted to rydberg, so the mean value of the energies and the typical error of the fit are printed in rydberg.

With several structures, the transition pressures are printed

```
 TRANSITION PRESSURE P=   12.000  BETWEEN   1   2
 TRANSITION PRESSURE P=   92.000  BETWEEN   2   3
```

They are found on the grid of `npts` pressures between 0 and `pmax`, so their precision is `pmax`/(`npts`−1).

With the Birch equation of state and $B_0' < 4$ the pressure has a maximum at high compression, and the structure becomes mechanically unstable ($B < 0$). If the maximum pressure of the plot is larger than that maximum, a warning is written and the structure is not considered at higher pressures.

The program also writes the following files

- `replay_eqst.dat`, a copy of the data in the current format, with comments, that can be used as a data file;
- `eofv_com.gp` and `eofv_dat.gp`, for the plot of the calculated and fitted energies per formula unit (in eV) as a function of the volume per formula unit;
- `eqst_com.gp` and `eqst_dat.gp`, for the plot of the volume of the most stable structure as a function of pressure, relative to the equilibrium volume of the structure with the lowest energy. Only written if there is more than one structure.

The plots are seen with `gnuplot`

```
cpw2000-5.x.y/WORK$ gnuplot eofv_com.gp
```

```
cpw2000-5.x.y/WORK$ gnuplot eqst_com.gp
```

or with `python` and `matplotlib`, as the program also writes the equivalent scripts `eofv_plot.py` and `eqst_plot.py`, which read the same `*_dat.gp` data files,

```
cpw2000-5.x.y/WORK$ python3 eofv_plot.py
```

```
cpw2000-5.x.y/WORK$ python3 eqst_plot.py
```

The `*_com.gp` and `*_plot.py` files can be edited by hand, for example to obtain a pdf file (see the commented lines in those files).

The original code was written by David M. Wood in the early eighties for the Murnaghan fit, and the Birch fit was added in 1989. It has been modified many times since then.

9.2 Remove gnuplot commands from 2D plot files

The 2D plot files are in the gnuplot format, with plotting directives. To remove the directives one can run `Tools/convert_2Dplot.f90`

9.3 Crystalography from cpw.in

Doing a crystalographic analysis after a self-consistent calculation may be too late. With `Tools/cpwin_geom.f90` the analysis may be done from the `cpw.in` file. Read the crystallography sub-section of the post-processing section to see how it works.

9.4 Coulomb potential

The Coulomb potential for hydrogen can be generate by `Tools/h_pot_kb.f90`

9.5 Graphical interface for analysis of band structure

This is a very useful tool. From a file with the relevant data it generates a very powerful GUI. There is an earlier version `BandInfoUi` based on the `grace` plotting package, which is similar to the most recent package and therefore there is no need to describe in detail.

The most recent package is the `QtBandViewer`. It is a `python` code that uses the `Qt` user interface framework (`https://www.qt.io`).

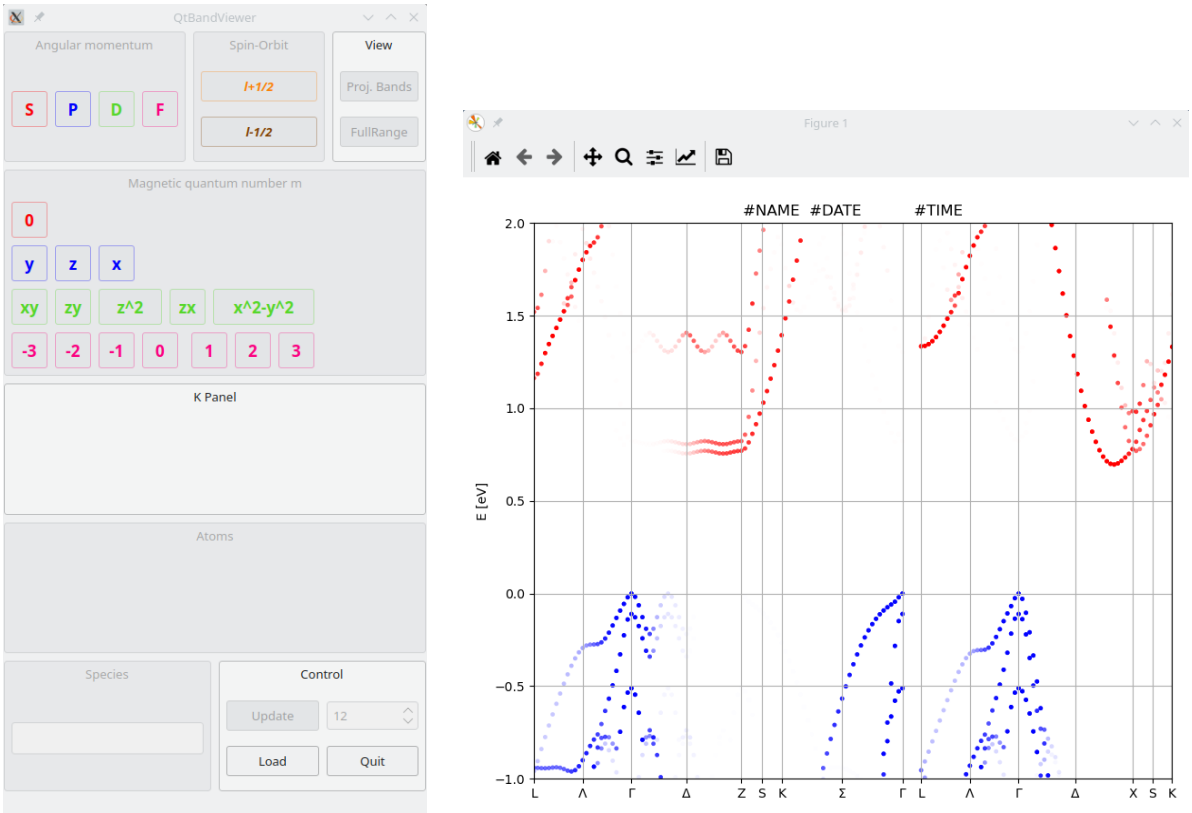
To install follow the instructions on `Tools/QtBandViewer/AAAREADME`. It is strongly recommended to create a python virtual environment under a working directory, and copy files there. By some obscure reasons (the writer of the guide does not use python) the python files should be copied. To use the tool you will need access to a `.bv` file format, by default named `BAND_SO.DAT.bv` or `BAND.DAT.bv` with the band energies and the `BAND_LINES.DAT` file used to generate those files. Those data files can be huge so avoid copying them.

As an example we will use the files generated from the structure file `Structures/Supercells/cpw_Si6Ge6_Si(001).in` with unfolding. The `BAND_LINES.DAT` was copied from `Structures/Supercells/band_lines_Si6Ge6_Si(001).dat`

once the code is installed, run it

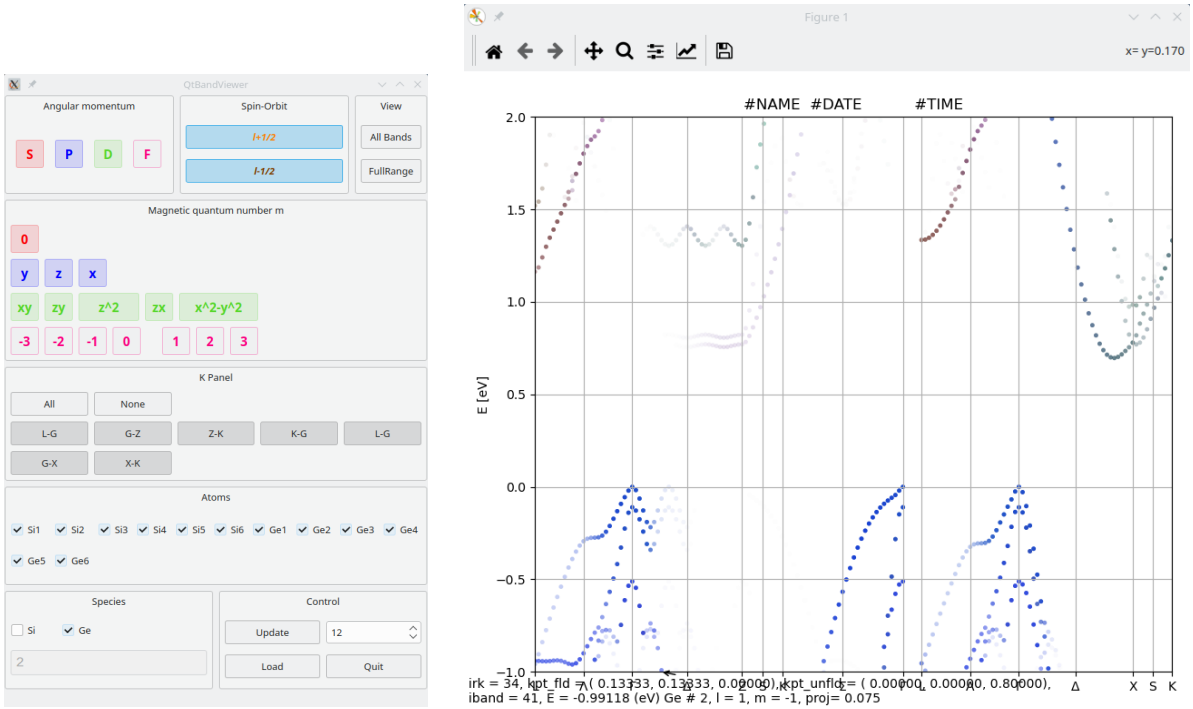
```
cpw2000-5.x.y/WORK/QtBV$ python3 QtBandViewer.py
```

The initial window is shown in the left of the figure. Press the `Load` button, and a new window shown on the right will display the unfolded band structure. Notice how the bottom of the conduction band appear to be in the Γ–X direction (parallel to the surface) and there are flat bands in the epitaxial growth direction Γ–Z.



If now we deselect Si leaving only the Ge contributions, and select the s, p and d angular momenta, we see that "dots" at the bottom of the band are still dark, but that the flat band

almost disappears. This indicates that the bottom of the conduction band has a contribution from the Ge layers and there is a localized band in the Si layer.



Reselecting Si and deselecting Ge and keeping the s, p and d angular momenta, we confirm that the flat band is indeed mostly Si, the bottom of the conduction band has a Si contribution that is weaker than the Ge contribution. It is also apparent that for the valence band at the unfolded Γ there is a band below the top and split-off bands that has a stronger Si content. For larger supercell periodicities it would converge to the band alignment between Si and strained Ge.

